

South African Road Running, Walking and Cycling Event Management System

Revised API contract based on the supplied RaceDay ERD and SQL Server schema. Unsupported fields from the earlier plan have been removed.

Convention	Value
API base	/api/
Primary roles	Admin, Organiser, Participant
Response format	JSON
Database	RaceDayDB — SQL Server
Endpoint style	Resource-oriented REST routes using HTTP methods and consistent naming

Key database alignment

The API maps to tblUsers, tblAdmin, tblOrganiser, tblParticipant, tblEvents, tblWeather, tblRoutes, tblCategory, tblRegistrations and tblResults.

Important: There is no direct Event → Result or Category → User relationship. Results are linked through Registration; Registration links Participant and Category.

Cardinalities used by the ERD

Relationship	Cardinality
Users → Admin	1 : 0..1
Users → Organiser	1 : 0..1
Users → Participant	1 : 0..1
Organiser → Events	1 : 0..*
Event → Weather / Routes / Categories	1 : 0..*
Route → Categories	1 : 0..*
Participant → Registrations	1 : 0..*
Category → Registrations	1 : 0..*
Registration → Result	1 : 0..1

API Conventions, Authentication and User Profile

Protected ownership must be derived from the authenticated request. Clients should not be trusted to supply another user's identity for ownership-sensitive operations.

Method	Purpose
GET	Retrieve resources or collections
POST	Create resources
PATCH	Partially update resources
DELETE	Remove/cancel resources where rules allow

1. Authentication

HTTP	Route	Purpose	Role	Request body	Response
POST	/api/auth/register	Create a user account. Normal public registration may create the participant profile.	Public	{"fullName","email","password","phoneNumber"}	201; 400
POST	/api/auth/login	Authenticate a registered user and establish a session/token.	Public	{"email","password"}	200; 401
POST	/api/auth/logout	End the current authenticated session.	Authenticated	None	204; 401

2. User Profile

HTTP	Route	Purpose	Role	Request body	Response
GET	/api/users/me	Return the current user's profile.	Authenticated	None	200; 401
PATCH	/api/users/me	Update editable user details.	Authenticated	{"fullName"?,"phoneNumber"?,"email"?"}	200; 400; 409
GET	/api/users/{id}	View a user profile where permitted.	Organiser / Admin	None	200; 403; 404

3. Events and 4. Event Categories

tblEvents contains EventID, OrganiserID, EventName, EventDate, Location and Description. The SQL schema has no Event Status column.

HTTP	Route	Purpose	Role	Request body	Response
GET	/api/events	List events; optional date/location filters.	Public	None	200
GET	/api/events/{id}	Return one event and related route/category information where available.	Public	None	200; 404
POST	/api/events	Create an event; OrganiserID comes from authenticated ownership.	Organiser / Admin	{"eventName","eventDate","location","description"?"}	201; 400
PATCH	/api/events/{id}	Update event fields.	Organiser / Admin	{"eventName"?", "eventDate"?", "location"?", "description"?"}	200; 403; 404
DELETE	/api/events/{id}	Remove/cancel where rules permit.	Organiser / Admin	None	204; 403; 404; 409

tblCategory contains EventID, RouteID, CategoryName, Gender, MinimumAge and MaximumAge. Fee, capacity, distance and ageGroup are not database fields.

HTTP	Route	Purpose	Role	Request body	Response
GET	/api/events/{eventId}/categories	List categories for an event.	Public	None	200; 404
GET	/api/categories/{id}	Return one category.	Public	None	200; 404
POST	/api/events/{eventId}/categories	Create a category and link it to a route belonging to the same event.	Organiser / Admin	{"routeId","categoryName","gender"?", "minimumAge"?", "maximumAge"?"}	201; 400; 404; 409
PATCH	/api/categories/{id}	Update category fields.	Organiser / Admin	{"routeId"?", "categoryName"?", "gender"?", "minimumAge"?", "maximumAge"?"}	200; 400; 404
DELETE	/api/categories/{id}	Remove when registrations do not prevent deletion.	Organiser / Admin	None	204; 404; 409

5. Event Registrations

tblRegistrations links tblParticipant to tblCategory and stores RegistrationDate, BibNumber and RegistrationStatus. ParticipantID should be derived from the authenticated participant.

HTTP	Route	Purpose	Role	Request body	Response
POST	/api/events/{eventId}/categories/{categoryId}/registrations	Register the authenticated participant. Validate that the category belongs to the supplied event.	Participant	{"bibNumber"}	201; 400; 404; 409
GET	/api/users/me/registrations	List the current participant's registrations.	Participant	None	200; 401
GET	/api/events/{eventId}/registrations	List registrations for an event.	Organiser / Admin	None	200; 403; 404
GET	/api/registrations/{id}	Return one registration; participant may access only their own.	Participant (own) / Organiser / Admin	None	200; 403; 404
PATCH	/api/registrations/{id}	Update registration status where permitted.	Organiser / Admin	{"registrationStatus"}	200; 400; 404
DELETE	/api/registrations/{id}	Cancel/remove registration where rules permit.	Participant (own) / Organiser / Admin	None	204; 403; 404; 409

Database rule

Column	Allowed values
RegistrationStatus	Registered, Cancelled, Completed
BibNumber	UNIQUE
ParticipantID / CategoryID	Required foreign keys

6. Results

tblResults contains ResultID, RegistrationID, FinishTime, Position and ResultStatus. RegistrationID is NOT NULL and UNIQUE, so one registration can have zero or one result. StartTime and Pace are not in the SQL schema.

HTTP	Route	Purpose	Role	Request body	Response
GET	/api/events/{eventId}/results	List results for registrations belonging to an event.	Public	None	200; 404
GET	/api/users/me/results	Show the current participant's result history.	Participant	None	200; 401
GET	/api/registrations/{registrationId}/result	Return the result for a registration.	Participant (own) / Organiser / Admin	None	200; 403; 404
POST	/api/registrations/{registrationId}/result	Create the single result permitted for a registration.	Organiser / Admin	{"finishTime"?, "position"?, "resultStatus"?"}	201; 400; 404; 409
PATCH	/api/results/{id}	Correct/update an existing result.	Organiser / Admin	{"finishTime"?, "position"?, "resultStatus"?"}	200; 400; 404

Result rules

Column	Rule
ResultStatus	Finished, DNF, DNS, Disqualified
Position	NULL or greater than 0
FinishTime	TIME; nullable
RegistrationID	UNIQUE — maximum one result per registration

The API deliberately excludes startTime and pace because those fields do not exist in tblResults.

7. Routes and 8. Weather

tblRoutes contains RouteID, EventID, RouteName, DistanceKM and Description. Elevation, start/end points and map URL are not database fields.

HTTP	Route	Purpose	Role	Request body	Response
GET	/api/events/{eventId}/routes	List routes for an event.	Public	None	200; 404
GET	/api/routes/{id}	Return one route.	Public	None	200; 404
POST	/api/events/{eventId}/routes	Create a route for an event.	Organiser / Admin	{"routeName","distanceKM","description"?"}	201; 400; 404
PATCH	/api/routes/{id}	Update route information.	Organiser / Admin	{"routeName"?", "distanceKM"?", "description"?"}	200; 400; 404
DELETE	/api/routes/{id}	Remove a route when dependent categories do not prevent deletion.	Organiser / Admin	None	204; 404; 409

Weather

tblWeather contains WeatherID, EventID, WeatherDate, Temperature, Conditions and WindSpeed. Humidity and ForecastTime are not database fields.

HTTP	Route	Purpose	Role	Request body	Response
GET	/api/events/{eventId}/weather	List weather records for an event.	Public	None	200; 404
POST	/api/events/{eventId}/weather	Add a weather record.	Organiser / Admin	{"weatherDate","temperature"?", "conditions"?", "windSpeed"?"}	201; 400; 404
PATCH	/api/weather/{id}	Update an existing weather record.	Organiser / Admin	{"weatherDate"?", "temperature"?", "conditions"?", "windSpeed"?"}	200; 400; 404

9. Administrative User Management and Final Alignment

The schema has separate role tables — tblAdmin, tblOrganiser and tblParticipant. It does not contain a Role table or RoleID column, so the previous generic roleId endpoint has been removed.

HTTP	Route	Purpose	Role	Request body	Response
GET	/api/users	List registered users for administration.	Admin	None	200; 403
PATCH	/api/users/{id}/status	Change AccountStatus.	Admin	{"accountStatus"}	200; 400; 404

Schema-to-API alignment

SQL table / constraint	API treatment
tblUsers	Authentication, profile and administrative user management.
tblAdmin / tblOrganiser / tblParticipant	Separate role/profile records linked to tblUsers with 1 : 0..1 cardinality.
tblEvents	Organiser-owned event CRUD; no status field.
tblWeather	WeatherDate, Temperature, Conditions, WindSpeed only.
tblRoutes	RouteName, DistanceKM, Description only.
tblCategory	EventID, RouteID, CategoryName, Gender, MinimumAge, MaximumAge only.
tblRegistrations	ParticipantID, CategoryID, RegistrationDate, BibNumber, RegistrationStatus.
tblResults	RegistrationID, FinishTime, Position, ResultStatus; one result maximum per registration.
Composite FK	tblCategory (RouteID, EventID) references tblRoutes (RouteID, EventID), ensuring the route belongs to the same event.

HTTP status convention: 200 OK = successful retrieval/update; 201 Created = successful creation; 204 No Content = successful removal; 400 Bad Request = invalid input; 401 Unauthorized = invalid/missing authentication; 403 Forbidden = not permitted; 404 Not Found = missing resource; 409 Conflict = constraint or business-rule conflict.

Implementation principle: keep routes, HTTP methods, roles, request fields and response behaviour synchronized with the SQL schema and ERD. Validate ownership and permissions on the server.